HIRAL SARKAR · PGA 45 CAPSTONE

Resolvia, from break to verified resolution

I built Resolvia to take a post-trade break from detection to a verified fix. This guide walks through how I built the data, the models and the agents, and what the results show. Every idea has a picture.

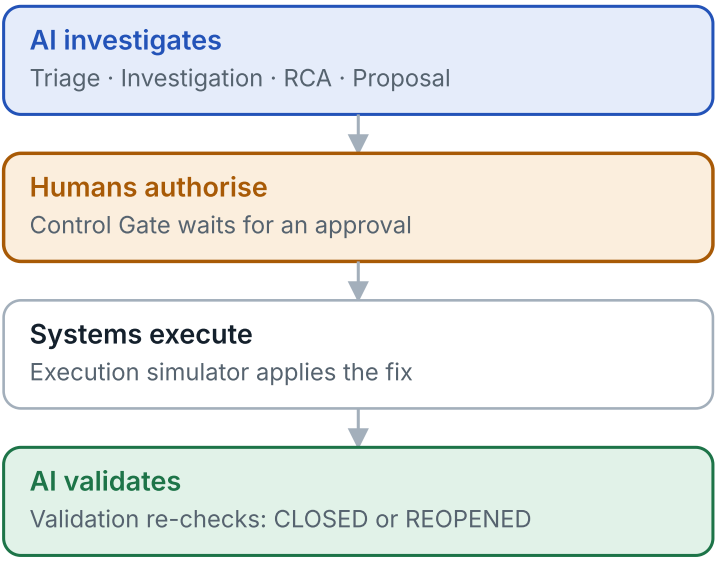
AI investigates | Humans authorise | Systems execute | AI validates

Idea | Architecture | Data | Modeling | Agents | R

01 · THE IDEA

Matching finds the break. Resolvia resolves it.

A reconciliation engine asks “does A equal B?” and raises a break when it doesn't. That part is solved: DTCC reports 98.4% same-day equity matching. The hard, manual part comes next: finding evidence, working out why, deciding the fix, getting sign-off and proving it worked. Resolvia automates that chain for cash-equity breaks, and keeps a human in charge of anything that moves money.



The one line to remember. Blue is AI, amber is human control, green is verification.

Scope. Indian cash equities (T+1, INR) · 4 break families · 20 root causes · a corporate-action specialist case. Out of scope: derivatives, FX, lending, real payment systems.



Resolvia starts after Level 0. It never rebuilds matching.

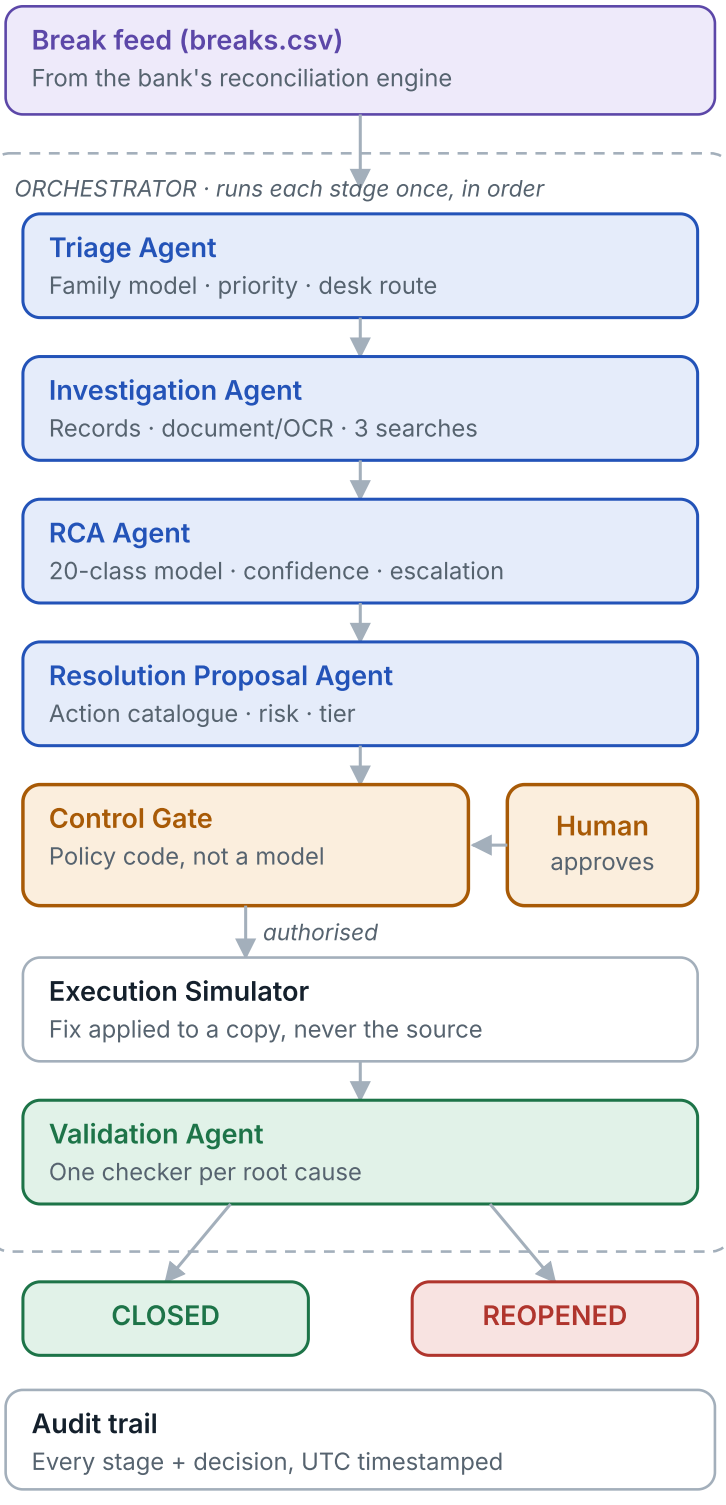
Where Resolvia sits. Level 0 is upstream; Levels 1 and 2 are the baselines it is measured against.

---

02 · ARCHITECTURE

Five agents, one gate, one orchestrator

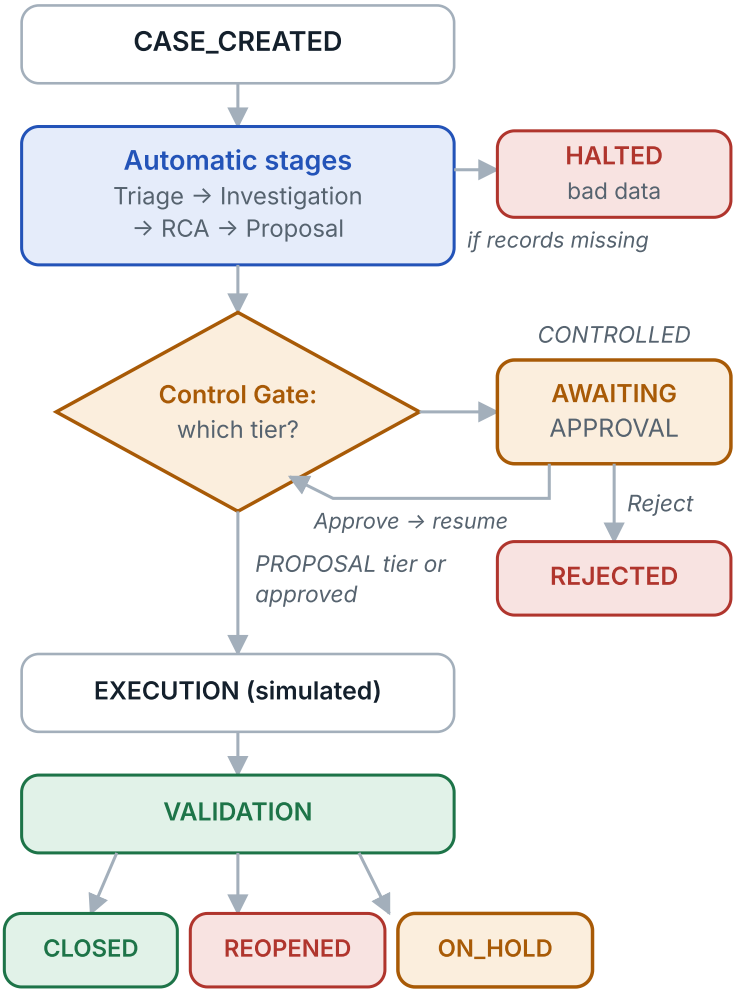
The Orchestrator runs the agents in order and owns exactly one decision: halt or proceed, based on the gate's answer. Every agent has a fixed authority limit. Only the Control Gate, which is plain policy code, can let an action through, and only a recorded human approval opens it for consequential actions.



One break, top to bottom. The dashed box is the Orchestrator; everything inside it runs automatically until the gate.

The case lifecycle

Because a program can't sit and wait for a person, the Orchestrator works in two phases: `create_case()` runs every automatic stage and pauses at the gate; `resume_case()` continues once a human has recorded a decision.

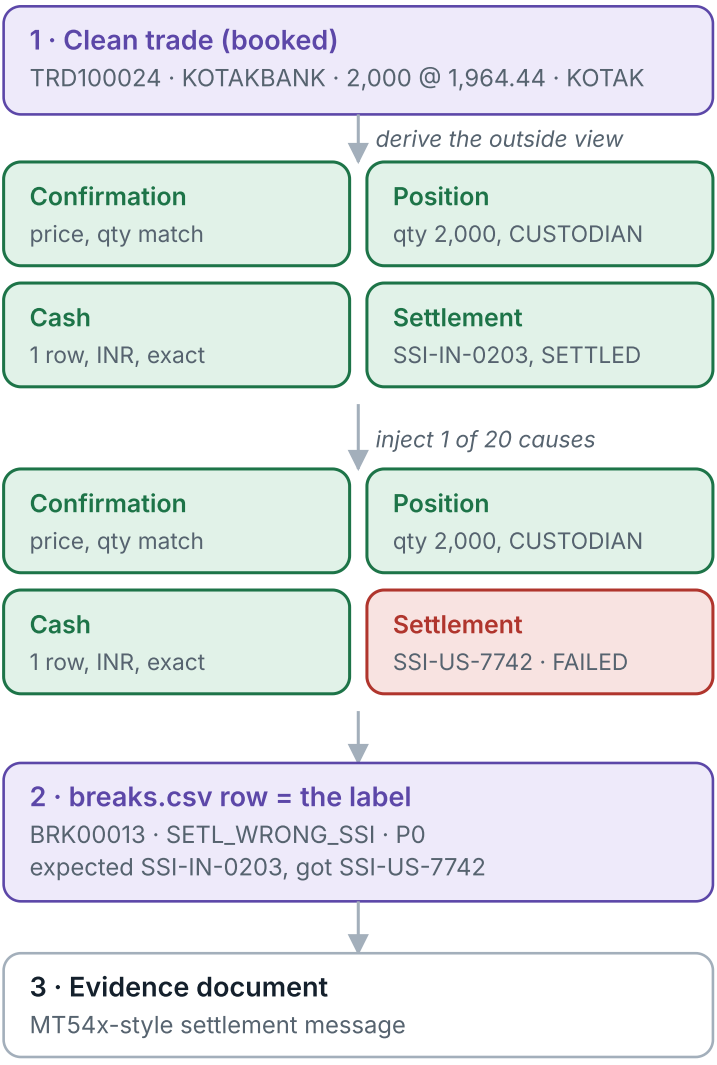


Every status a case can reach. A REOPENED case goes back for re-investigation, never a blind retry.

A synthetic bank, built so the answer is known

No public dataset of real trade breaks exists; trade data is confidential everywhere. So, like PaySim for fraud, I simulated clean operations, injected one known fault per break, and kept the ground truth. Everything is seeded (`random.seed(42)`), so every run is identical.

6,000 trades	**3,000** breaks	**150** per root cause
2,160 documents	**30** scanned images	



How one break is born: start clean, perturb exactly one table, write the label row, write the document.

A note on the data: I set the break rate at 50%, far higher than real life (about 1 to 5%), to give each of the 20 classes 150 examples. Trade breaks change the trade's own `confirmed_*` columns, so the model never has to read the label to find them.

The taxonomy: 4 families, 20 root causes

Family is for routing to the right desk. Root cause is what the model predicts. Severity P0 means settlement or critical risk.

POSITION	6
P0	Missing booking
P1	Wrong quantity
P2	Timing difference
P1	CA incorrect entitlement
P1	CA mandatory not reflected
P1	CA voluntary decision

CASH	4
P0	Missing entry
P1	Duplicate entry
P1	Wrong amount
P2	FX / currency

SETTLEMENT	4
P0	Wrong SSI
P0	Missing instruction
P1	Late counterparty
P2	Incorrect date

TRADE	6
P0	Confirmation missing
P1	Quantity mismatch
P1	Counterparty ID mismatch
P1	Confirmation disagreement
P2	Price mismatch
P2	Date mismatch

The documents

Every corporate-action, settlement and trade break gets a document, plus a 25% sample of position and cash breaks. Each family gets the format a bank would really receive:

Corporate action **SWIFT MT564 notice**

```
:22F::CAEV//BONU
:22F::CAMV//MAND   ← mandatory or VOLU
:22F::CAON//3:1
```

Settlement **SWIFT MT54x message**

```
:16R:GENL
:20C::SEME//…
:95Q: SSI / custodian
```

Trade **NSE contract note**

```
UCC · SEBI reg no.
Qty · Price · STT · GST
(the counterparty's version)
```

Position / Cash **Ops messages**

```
email · note · chat · WhatsApp
"RIL" / "Reliance Industries"
dates in 3 formats
```

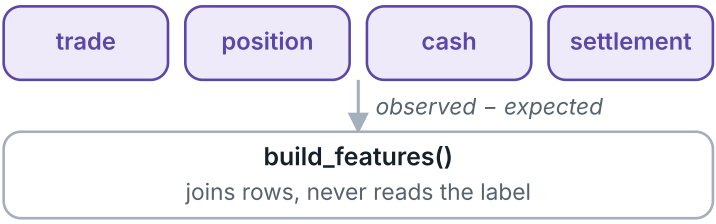
---

04 · MODELING

Two Random Forests on 15 features

The family model (4 classes) routes in Triage. The root-cause model (20 classes) diagnoses in RCA.

Both read the same 15 features, each measuring how far the outside world is from the booking.



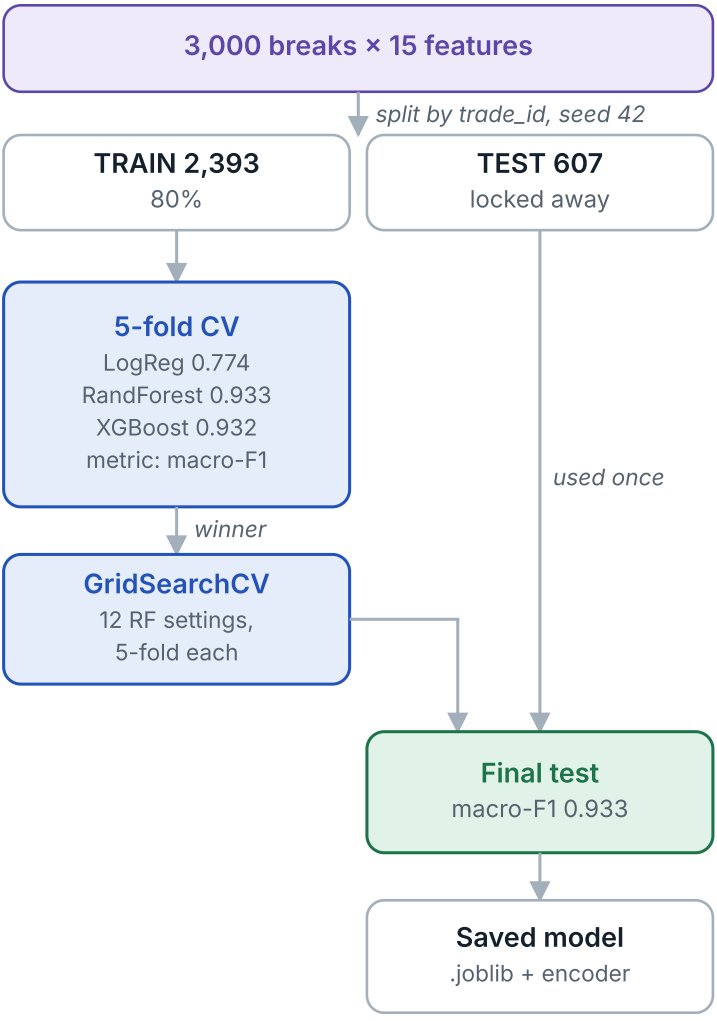
Features come from joining the four tables. The label columns are never used as inputs.

Example: the 15 features for the wrong-SSI break above. Only the cells that differ from a clean trade light up.

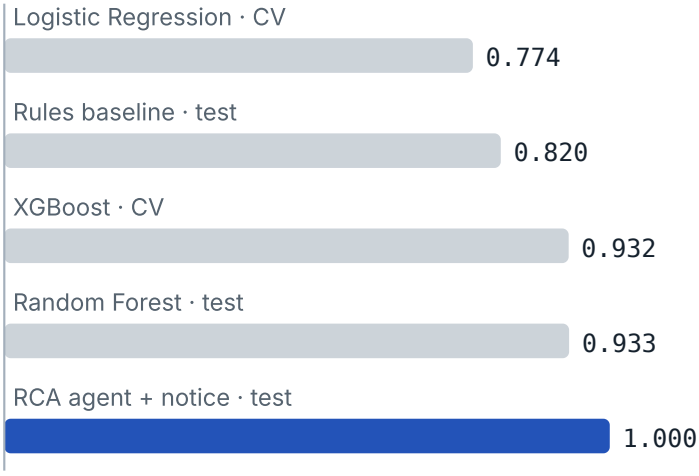
qty_delta_pct	0	position_date_delta	0
cash_row_count	1	cash_amount_delta_pct	0
cash_currency_mismatch	0	settlement_date_delta	0
ssi_mismatch	1	instruction_missing	0
settlement_failed	1	settlement_pending	0
price_delta_pct	0	confirmed_qty_delta_pct	0
date_mismatch	0	counterparty_mismatch	0
confirmation_missing	0		

Why root cause, not family, is the target

I tested both first. Family scored a perfect 1.000 with tree models: it's basically “which table has a non-zero difference”, so there's nothing to compare or tune. Root cause is a real 20-class problem, so it became the graded model. Family is still reported by mapping each predicted cause to its family (also 1.000).



Model comparison and tuning. The test set is touched exactly once, at the end.



Bars start at 0. Macro-F1 across 20 root causes.

Macro-F1 gives every class equal weight, so one failing class can't hide behind good ones.

How 0.933 is made

18 classes are perfect. The two that aren't are a trap I built on purpose. (18 × 1.00 + 0.66 + 0.00) ÷ 20 = 0.933.

pos missing booking	1.00	pos wrong quantity	1.00
pos timing difference	1.00	ca incorrect entitlement	1.00
ca mandatory not reflected	0.66	ca voluntary decision	0.00
cash missing entry	1.00	cash duplicate entry	1.00
cash wrong amount	1.00	cash fx input difference	1.00
setl wrong ssi	1.00	setl late counterparty	1.00
setl missing instruction	1.00	setl incorrect date	1.00
trade price mismatch	1.00	trade quantity mismatch	1.00

trade date mismatch	1.00	trade counterparty id mismatch	1.00
conf missing	1.00	conf disagreement	1.00

The corporate-action trap

Mandatory and voluntary corporate actions were generated to look identical in the numbers. The model calls both “mandatory”. Only the notice tells them apart, and the actions are opposite.

	Mandatory split CA_MANDATORY_NOT_REFL.	**Voluntary tender** CA_VOLUNTARY_DECISION
Booked qty	500	500
Custodian qty	500	500
All 15 features	zero	zero
Model predicts	MANDATORY	MANDATORY
MT564 notice	CAMV//MAND	CAMV//VOLU
Right action	Apply ratio	HOLD for client

Numbers can't tell them apart. The document can.

This is the project's main argument: structured ML has a ceiling exactly where a business judgment lives.

So the RCA agent reads the notice. Its CAMV field overrides the model, which takes root-cause macro-F1 from 0.933 to 1.000 on the test set. A person still signs off every corporate action, because **confidence never decides authority.**

---

05 · THE AGENTS

Each agent: one question, one limit

An “agent” here is a Python module with one entry function, a fixed input and output, and a stated authority limit. They are specialists, not chatbots: each uses the cheapest reliable tool for its step, and the LLM is optional.

Triage
What is it, where does it go?
`No financial action`

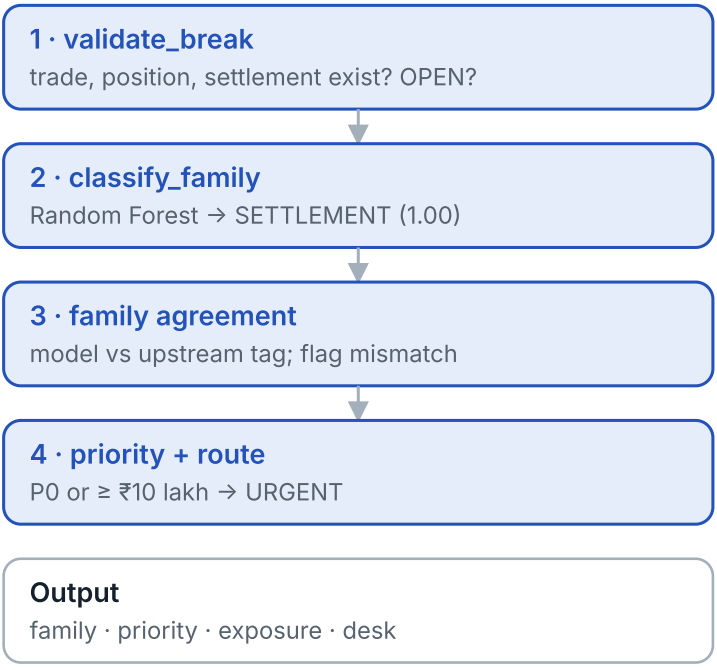
Investigation
What evidence exists?
`Read only`

RCA
Why did it happen?
`Recommends only`

Resolution Proposal
What should we do?
`Proposes only`

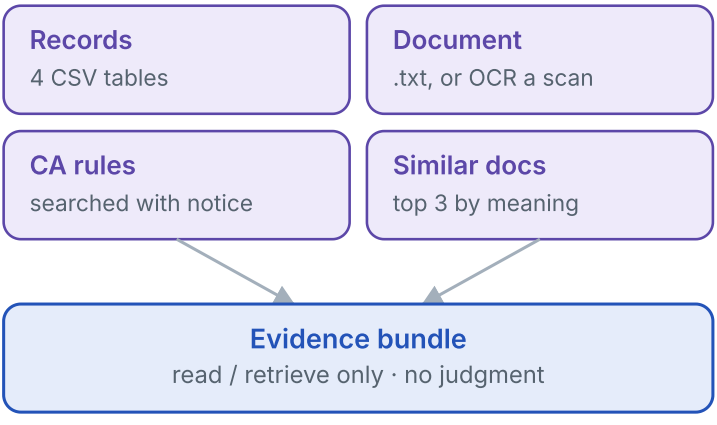
Validation
Did the fix work?
`CLOSE or REOPEN`

Triage Agent



Data check, family model, sanity check against the upstream tag, then priority and desk. Corporate actions always go to the CA specialist queue.

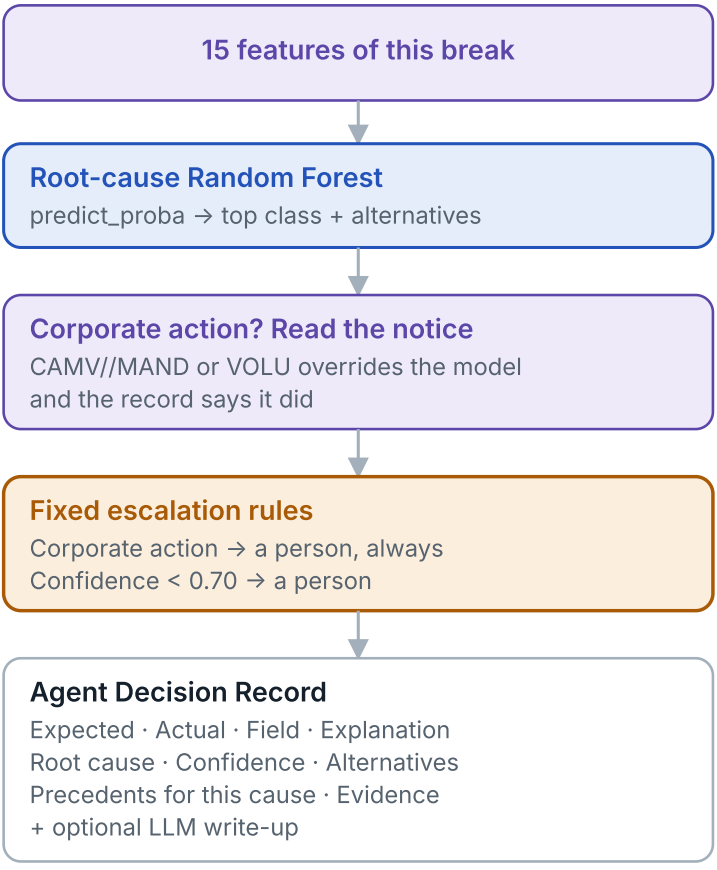
Investigation Agent



Never reads the generator's answer column

Gathers, never judges. It only sees what a reconciliation engine would hand over.

RCA Agent



The model diagnoses, the notice corrects it where it can, and fixed rules decide whether a person reviews it. Precedents are looked up for the diagnosed cause.

Resolution Proposal Agent

Once the cause is known, the fix is mechanical, so it is looked up in a fixed catalogue rather than invented by an LLM. Each entry has an action, an expected outcome and an authority tier. Risk is arithmetic: HIGH if P0 or exposure ≥ ₹10 lakh.

15 causes · ~75%
CONTROLLED_ACTION · human approval

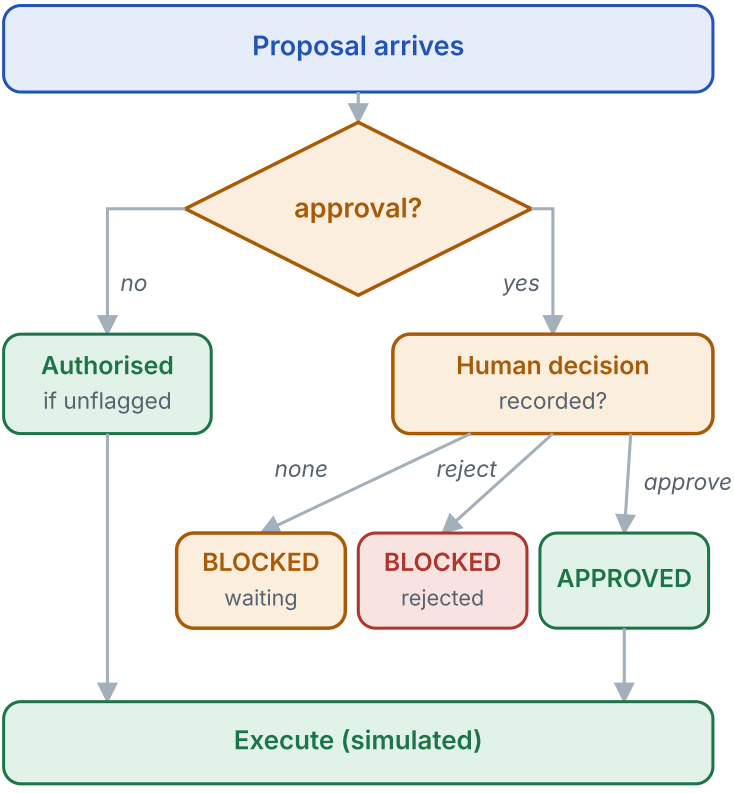
5 · ~25%
PROPOSAL

PROPOSAL tier (administrative): timing difference, late counterparty, settlement date, trade date, missing

confirmation. Everything that changes money, an SSI or a counterparty is CONTROLLED. Voluntary corporate action = **HOLD**. Even a PROPOSAL-tier step needs approval if RCA flagged the diagnosis.

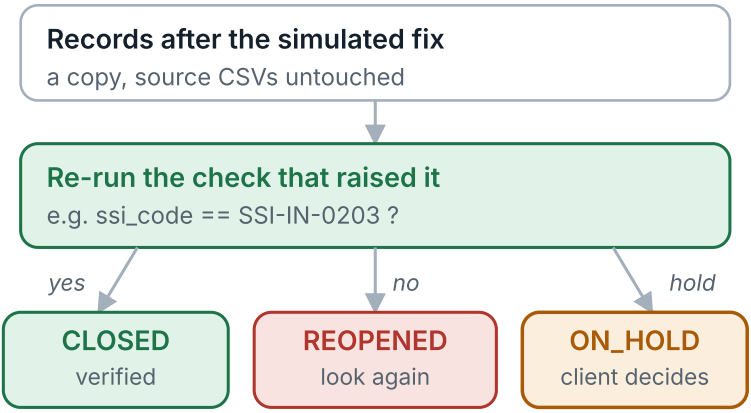
> **Two different flags.** RCA's `requires_human_decision` asks "is the diagnosis uncertain?". Resolution's `approval_required` asks "does the action need sign-off?". A confident diagnosis can still need approval.

Control Gate



record_decision() is the only door a human approval comes through; no code can approve itself. Result: 0 unauthorised executions.

Validation Agent



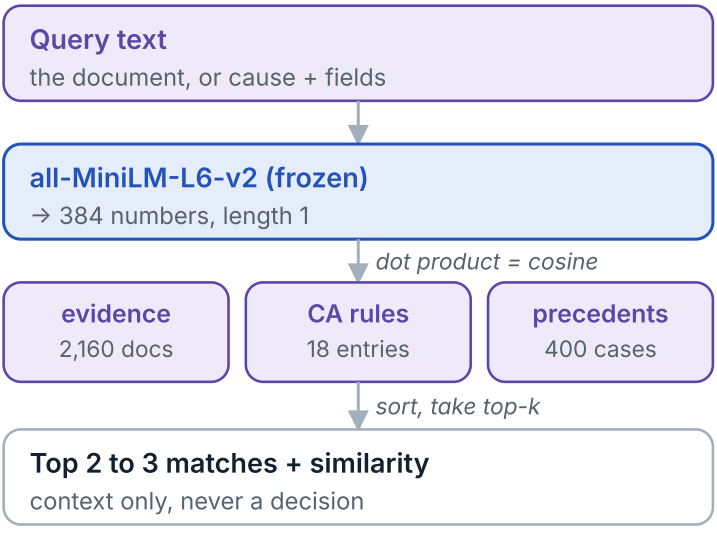
Doesn't trust the diagnosis: a wrong fix reopens
Gate blocked = nothing ran = stays OPEN

Validation re-runs the reconciliation on the field that broke. If the diagnosis was wrong, the wrong fix runs and this catches it.

---

06 · RETRIEVAL, OCR, LLM

Context comes from search, not memory



One frozen embedding model, three local indexes, plain NumPy. No vector database needed at this size.

Precedents

20 breaks per root cause were run through the real pipeline: 400 cases: 380 CLOSED and 20 ON_HOLD. The 20 on hold are voluntary corporate actions waiting for the client, which is the right outcome.

OCR

EasyOCR reads the 30 scans, ~99% accurate on key tokens (trade IDs, symbols, quantities). It runs only when a document exists solely as an image.

LLM (optional)

OpenRouter with a fallback list of free models. Writes the RCA narrative, extracts fields from emails, summarises uploads. It explains; it never decides.

---

07 · WALKTHROUGH

One P0 break, start to finish

BRK00013, SETL_WRONG_SSI: the payment went to the wrong settlement account and failed. This is the case I use to demo it.

Case created

Analyst opens the break; CASE_CREATED is logged.

Triage

Records OK · family SETTLEMENT (agrees) · P0 → URGENT · Settlements Desk.

Investigation

Settlement row: SSI-US-7742, FAILED, INCORRECT. MT54x document read. 3 similar documents found.

RCA

SETL_WRONG_SSI at ~100%. Expected SSI-IN-0203, actual SSI-US-7742. Not CA, confident → no forced review. 3 precedents pulled.

Proposal

"Correct the SSI and resubmit." CONTROLLED_ACTION, risk HIGH.

Gate blocks

No approval yet → AWAITING_APPROVAL. Nothing touched.

Human approves

Approve + note. Who and when are recorded; the case resumes.

Execution

On a copy: correct SSI, INSTRUCTED, SETTLED.

Validation

Re-checks ssi_code: now SSI-IN-0203 → CLOSED.

Audit

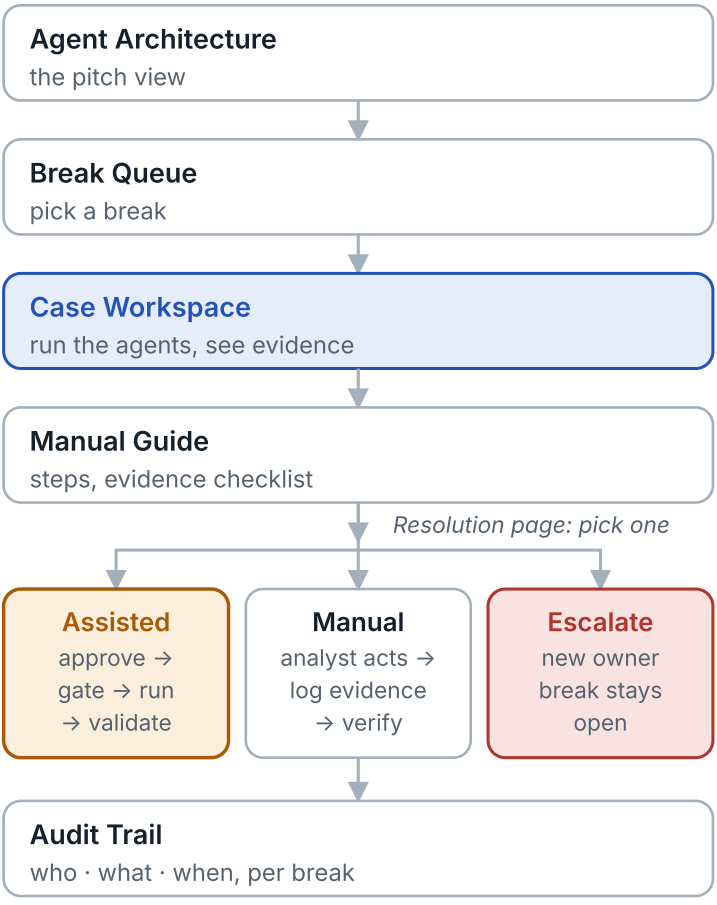
Every step above is in the trail with a timestamp and Operations ID.

---

08 · THE APP

Streamlit workbench for three people

Analyst: next action and evidence. Manager: workload, ageing, exposure. Auditor: who did what, when. The app calls the backend in-process through `backend_bridge.py`; the operator ID (default OPS-1047) is stamped on every event.



Also: Home (workload) and Manager Overview (exposure, ageing).

The demo path through the app. Escalation transfers ownership; it is never a closure.

Trying it yourself. The first analysis takes about 20 seconds while the models load. I'd start with BRK00013 end to end, then BRK00030, a voluntary corporate action, where the notice overrides the model and the case ends on hold. Cases reset when the server restarts.

09 · RESULTS

The numbers

1.000
RCA agent with the notice (test)

0.933
model alone (test)

0.820
rules baseline

77%
test breaks needing a person

572 / 35
test breaks closed / on hold

0
unauthorised actions

~99%
OCR on scans

4.89
sources per break, manual

The three-way comparison

Level 1

Manual

Analyst opens about 5 evidence sources per break (4 tables, plus the document, plus the CA notice). A process proxy, not a made-up stopwatch time.

Level 2

Rules / RPA

If-else rules on the same 15 features: 0.820. Nails mechanical causes, fails on corporate actions.

Level 3

Resolvia

0.933 from the model, 1.000 once the notice is read, plus evidence assembly, precedents, enforced approval, simulated execution and a re-check before closing.

Business value formula (no invented dollar figure): capacity released = break volume × manual minutes per break ×

automation coverage × working days. Cost avoided = capacity × loaded analyst cost per hour.

10 · QUESTIONS

Limits and common questions

What my build doesn't do

- State lives in memory; it resets on restart.
- Execution is simulated on a copy; no real system is touched.
- Data is synthetic; scores outside the corporate-action classes are near-perfect because synthetic differences are clean.

Choices I made so it behaves like a real system

- **It never reads the answer.** The root-cause columns in breaks.csv are ground truth for training only. The app shows what the reconciliation engine reported until RCA has diagnosed the break.
- **Fixes follow the diagnosis.** Execution applies the fix for the predicted cause, and Validation re-checks the field that broke, so a wrong diagnosis reopens instead of closing.
- **The notice settles the trap.** RCA reads CAMV//MAND or VOLU from the MT564 and overrides the model when they disagree.
- **Flags reach the gate.** Even an administrative step needs approval if RCA flagged the diagnosis.
- **Voluntary means hold.** These end ON_HOLD, waiting for the client's election, not REOPENED.

Questions I get asked

▼ Why synthetic data?

No public trade-break dataset exists; it's confidential. I took the same approach as PaySim, because I needed known ground truth to measure anything.

▼ Why root cause instead of family as the target?

Family hit 1.000 for tree models, so there was nothing to compare or tune. Root cause is a real 20-class problem with a real gap between models.

▼ Why Random Forest over XGBoost?

Highest mean CV macro-F1 (0.933 vs 0.932, a tie in practice). Simpler, less tuning, and clean probabilities for the confidence and alternatives fields.

▼ Why macro-F1, not accuracy?

With 20 classes, macro-F1 weights each one equally, so a failing class like voluntary corporate actions is visible instead of averaged away.

▼ Where is the “agentic” part if the agents aren't LLMs?

Autonomous specialists with bounded authority, coordinated by an orchestrator, each using retrieval and tools. The LLM is one tool, used only where explaining across mixed evidence helps.

▼ What stops the AI from changing a payment?

The Control Gate. It's policy code; a recorded human approval is the only way in, and nothing can approve itself. Zero unauthorised executions.

▼ If the notice fixes it, why still send corporate actions to a person?

Because the model alone is confidently wrong on voluntary ones, and in real life a notice can be late, missing or a bad scan. Confidence must never decide authority.

▼ What does RAG do here?

Three things: related documents, corporate-action definitions (searched with the notice text), and past cases for the diagnosed cause. It supplies context; it never decides.

▼ How do you know a fix worked?

Validation re-runs the comparison that raised the break on the updated records. It does not trust the diagnosis, so a wrong fix reopens the case.

▼ How would this go to production?

Plug in the bank's break feed instead of the generator, real reference data (NSE Bhavcopy, corporate-action feeds), a database, sign-in, the bank's maker-checker system behind the gate, and validation against live records.

Repository map

`data/`
generators, taxonomy, features, 5 CSVs, 2,160 docs, 30 scans

`backend/agents/`
triage, investigation, rca, resolution, validation (+ ingestion)

`backend/`
orchestrator, control_gate, execution_simulator, audit, llm

`backend/models/`
training scripts + saved .joblib models

`backend/embeddings/`
MiniLM store + 3 indexes

`backend/knowledge/`
18 CA rules, 400 precedent cases

`backend/eval/`
rules baseline, end-to-end evaluation, gate demo

`backend/api/`
REST API: open a case, record a decision

`docs/`
the tour, diagrams, model report, EDA notebook, proposal

`frontend/`
Streamlit pages and backend bridge

Stack: Python, Streamlit, Plotly, pandas, scikit-learn, XGBoost (compared), sentence-transformers, EasyOCR,

FastAPI, OpenRouter. Live app: resolvia.streamlit.app · Code: github.com/hiralsarkar/Resolvia

Hiral Sarkar · github.com/hiralsarkar/Resolvia. Colours: blue = AI agent, amber = human control, green = verified, red = failure, violet = data.